

1. Data Representation

1.1. User-defined data types

- A data type constructed by the programmer using primitive data types.
- User-defined data types are used to create new data types that help to extend the flexibility of a programming language as the programmer can create data types based on the application's requirements.
- Importance of user-defined data types:
 - Code organization- Helps structure and group related data logically, making programs easier to read and maintain.
 - Reusability- Once defined, the data type can be reused multiple times, reducing redundant code and improving efficiency.
- Examples include:
 - Record
 - Set
 - Class
 - Object

Composite User-Defined Data Types

Composite user-defined data types have a definition referencing at least one other type.

- Record:** A composite data type that is a collection of related items which may or may not have different data types

```
TYPE TBook // Example
  DECLARE Name : STRING
  DECLARE Isbn : INTEGER
ENDTYPE
```

```
DECLARE Book1: TBook // Creating a variable
Book1.Name = "To Kill a Mockingbird" // Accessing properties
```

- Set:** A set stores a finite number of unique items in no particular order allowing the program to apply mathematical operations defined in set theory.

```
TYPE <set-name> = SET OF <type> //Declaration
DEFINE <var-name> (value1,value2,value3,...) : <set-name>
```

```
TYPE Numbers = SET OF INTEGERS
DEFINE EvenNumbers (2,4,6,8,10) : Numbers
```

- Classes:** In object-oriented programming (OOP), a class is the base for creating objects. It has methods and properties.

```
// Class definition
CLASS Bird
  PRIVATE Name: STRING
  PUBLIC PROCEDURE NEW(pName: STRING)
    Name ← pName
  ENDPROCEDURE
ENDCLASS
```

```
// Implementing inheritance
CLASS FlyingBird IMPLEMENTS Bird
  // Class info...
ENDCLASS
```

```
// Creating an object
<name> ← NEW <class_name>(<parameter1>, <parameter2>, ...)
MyPet ← NEW Bird("Kiwi")
```

Non-Composite Data Type

Data types that can be defined without referencing another type.

String and Char are both non-composite built-in types

Examples include:

- Integers
- Booleans
- Strings
- Enumerated Data Type:** A user defined non-composite data type that contains an ordered list of all possible values.

```
TYPE <name> : (<value1>, <value2>, ...)
TYPE Days: (Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday)
TYPE Numbers: (1..10) // Inclusive
DECLARE Today: Days // Declaration
Today ← Sunday
Today ← Today + 1 // Value is now Monday
```

- Pointer Data Type:** A user defined non-composite data type that is used to reference a memory location.

```
TYPE <Name> = ^<Type>
```

```
// Declaring a pointer type
TYPE PIntPointer: ^INTEGER
// Declaring a pointer variable
DECLARE MyPointer: PIntPointer
```

```
MyPointer^ ← 3 // Sets the value at the address to
MyPointer ← 3 // Pointer points to value at the address 3
MyPointer ← @<variable_name> // Makes pointer point to value stored at
```

1.2. File Organization

Serial

In serial files, records are stored one after another in chronological order. Records are appended to the next available spot.

- Benefits:**
 - No need to sort the list.
 - Data can be appended easily.

Sequential

In sequential files, records are stored one after another in the order of the key field. Records are inserted into the correct position. A new version of the file has to be created to update it.

- Benefits:**
 - Allows for batch processing.
 - Allows the data to be stored and sorted in a specific order.

Random

- In random files, records are stored in no particular order. Changes to the file are done directly. In random file organization, a unique key field is passed through a hash algorithm which produces the home location for the data. If that location is empty, the data is stored; otherwise, it relies on an overflow method to resolve the collision.
- A **collision** occurs when two different records produce the same hash value, meaning that the target location is already being used to store a different record.
- To handle collisions, random file organization uses one of two main hashing algorithms:
 - Open Hashing (Separate Chaining)
 - Mechanism: The records are not restricted to the primary positions available within the main file array. Instead, each slot (bucket) in the file contains a pointer to a separate, dynamically allocated linked list.
 - Handling Collisions: When a collision occurs, the new record is simply appended to the end of the linked list branching off from that specific home location. The record is stored outside the fixed limits of the main file array.
 - Finding a Record: The key is passed through the hash algorithm to locate the home bucket. The system then reads sequentially down the linked list attached to that bucket until the matching record is found.
 - Benefits: The file can handle an infinite number of collisions without ever running out of physical table slots, and deletions are cleaner because you only have to adjust linked list pointers.
 - Closed Hashing (Open Addressing)
 - Mechanism: All records must be stored directly inside the fixed boundaries of the main file structure itself. No external structures or auxiliary linked lists are allowed.
 - Handling Collisions: When saving a file, the algorithm searches for an alternative, empty slot (an open address) within the same file array using a specific mathematical probing routine:
 - Linear Probing: The system searches the file linearly (+1, +2, +3...) for the very next empty spot and saves the record there.
 - Quadratic Probing: The system searches intervals expanding quadratically ($+1^2, +2^2, +3^2...$) to find an empty spot and prevent data clustering.
 - Finding a Record: When finding a record, the system checks the home location. If it does not match, it starts from that current location and searches the array or the designated overflow area linearly step-by-step until the matching record is found or a completely empty slot is hit (signaling the record does not exist).
 - Benefits: Highly memory efficient since no extra pointer space or dynamic memory allocations are needed. It is fast as real-time processing is used.

File Access

Sequential Access

In sequential access, records are read in the exact order that they are stored in. It is read record by record until the target record is found or the key field value is exceeded.

- Usage:** This is used for both **serial** and **sequential** file organization.

Direct / Random Access

In random access, you can directly access the specific data needed without reading the preceding records.

- **Usage:** This is used in both **sequential** and **random** file organization.
- **Implementation:** In sequential file organization, an index of all the key fields is used to jump to the data, while a hashing algorithm (using either open or closed hashing rules) is used in random file organization.

1.3. Floating-point numbers, representation and manipulation

Floating point representation

- **Mantissa:** Represents the numbers. If we increase the bits allocated to the mantissa, we increase accuracy.
- **Exponent:** Represents the number of times the decimal place needs to move. If we increase the bits allocated to exponent, we increase the range.

$$M \times 2^E$$

$$M = \text{Mantissa}$$

$$E = \text{Exponent}$$

Converting from Binary Digit to Positive Float

$$\text{Mantissa} = 01101000 \text{ Exponent} = 0011$$

1. Write the binary number with the decimal point

$$0.1101$$

2. Multiply with 2 to the power of the exponent (Move the decimal point the same number of places as the exponent)

$$0.1101 \times 2^3 \rightarrow 1101.1$$

3. Convert to decimal by multiplying

$$(4 \times 1) + (2 \times 1) + \left(\frac{1}{2} \times 1\right)$$

Therefore the answer is 6.5

Converting from Float to Binary Digit

$$6.5$$

- Make the float into an integer by multiplying by factors of 2 till it becomes a whole number.

$$6.5 = \frac{13}{2}$$

- Now multiply the denominator by multiples of 2 till it is larger than the numerator.

$$\frac{13}{2} \xrightarrow{\boxed{1}} \frac{13}{4} \xrightarrow{\boxed{2}} \frac{13}{8} \xrightarrow{\boxed{3}} \frac{13}{16}$$

- So we get exponent as 3 which in binary is 0011. Now for mantissa, we slowly split $\frac{13}{16}$ into factors of $\frac{1}{2}$.

$$\frac{13}{16} = \frac{1}{16} + \frac{12}{16} = \frac{1}{16} + \frac{6}{8} = \frac{1}{16} + \frac{3}{4} = \frac{1}{16} + \frac{1}{4} + \frac{2}{4} = \frac{1}{16} + \frac{1}{4} + \frac{1}{2}$$

- Put these back into the mantissa table

Mantissa: 01101000

Exponent: 0011

Converting from Binary Digit to Negative Float

By taking two's complement on a positive binary float, converts it gets converted into the negative float.

- Do not take two's complement of the exponent.

$$01110000 \ 0010 \rightarrow 10010000 \ 0010$$

+3.5 became -3.5

Normalization

For a positive binary digit, the first two bits need to be different. (Removing extra zeros before in between the decimal and the first 1 digit).

$$0.0110000 \ 0011 \rightarrow 1.100000 \ 0010$$

Benefits:

- Avoids two of the same number being represented differently.
- Maximizes the potential precision.
- Allows us to store the maximum range of numbers in the minimum number of bits.

Negative binary numbers

- For negative binary digit, the first bit needs to be 1.

- The mantissa are shifted until we arrive at 1.0, then the exponent must be changed to reflect the change.

For example:

1.1101100 --> 1010

1.0110000 --> 1000

Unlike positive normalisation, the number of changes are subtracted from the exponent.

Important values

Largest positive number	0111 1111 0111
Smallest positive number	0000 0001 1000
Smallest normalized positive number	0100 0000 1000
Largest magnitude negative number	1000 0000 0111

Errors

- **Rounding Error:** When numbers can't be represented exactly in binary, an approximate value is used which is either more or less than the needed value. When these numbers are then used in operations, the inaccuracies get added to the point where they are visible in the final answer. Like $0.1 + 0.2 = 0.300000000000004$
- **Underflow:** Underflow occurs when a number is too close to zero to be accurately represented in binary, leading to a loss of precision.
- **Overflow:** Overflow happens when a number's magnitude exceeds the limit that can be represented in a given number of bits.